

CENTRO REGIONAL UNIVERSITARIO CÓRDOBA IUA



Arquitectura Orientada a Servicios – Ingeniería Informática

Traefik – API Gateway con Load Balancing en Docker

Profesor: Mariano Garcia Mattio

Grupo: 5

Fecha de Entrega: 28/04/2026

Integrantes (Ingeniería en Informática)

- **Garibaldi, Bruno** | D.N.I.: 44.740.709
- **Macedo, Bautista** | D.N.I.: 44.780.197
- **Rodeyro, Agustín** | D.N.I.: 44.672.538

Índice

Índice	1
1. Resumen ejecutivo	2
2. Objetivo y alcance	2
Objetivo	2
Alcance	2
3. Desarrollo	3
3.1 ¿Qué es Traefik?	3
3.2 Casos de uso de Traefik	3
3.2.1 Traefik como Balanceador de Carga	3
3.2.2 Traefik como Reverse Proxy	4
3.2.3 Observabilidad	4
3.2.4 Seguridad	4
3.3 ¿Cuáles son sus componentes?	5
3.4 Comparativa con sistemas alternativos a Traefik	5
3.4.1 Traefik vs Nginx vs HAProxy	6
3.5 Desventajas y Limitaciones.	6
4. Arquitectura	6
4.1 Diagrama Arquitectura	6
4.1.1 Visualización en el dashboard de Traefik	7
4.2 Explicación Arquitectura	7
5. Implementación	7
5.1 docker-compose.yml	7
5.1.2 Balanceo de Carga	8
5.1.3 Middleware	8
6. Pruebas y evidencias	9
6.1 Funcionamiento Routers	9
6.1.1 Funcionamiento Routers por Path	9
6.1.1.1 /api	9
6.1.1.2 /static	9
Creamos un 6.1.2 Funcionamiento Routers por Puerto	9
6.2 Funcionamiento Middleware	10
6.3 Funcionamiento Balanceo de Carga	10
6.3.1 Prueba con CURL y diferenciación por IP	10
7. Problemas/Soluciones - Extras	11
7.1 Autodescubrimiento y File Provider: configuración dinámica sin reinicio	11
7.2 HealthChecks	12
7.3 Enrutamiento a distintos host con balanceo de carga por prioridad	13
8. Conclusión	14
9. Anexo	14
10. Referencias	14

1. Resumen ejecutivo

El siguiente trabajo documenta el desarrollo e implementación de una arquitectura orientada a servicios utilizando Traefik como reverse proxy y balanceador de carga en un entorno con Docker.

La implementación consiste en una estructura desplegada mediante Docker Compose compuesta por tres elementos:

1. Traefik como único punto de entrada (puerto 80)
2. Cluster de tres réplicas del servicio whoami al que se aplica balanceo de carga
3. Servidor de contenido estático basado en Nginx

Los servicios internos no exponen puertos al host, siendo Traefik el único punto de acceso externo.

Para validar el funcionamiento se ejecutaron tres tipos de pruebas. Primero pruebas de ruteo mediante curl para verificar que las rutas /api y /static son correctamente dirigidas a sus servicios correspondientes. Por otro lado, se realizó una prueba de balanceo de carga ejecutando requests sucesivos contra /api/whoami, evidenciando que Traefik distribuye el tráfico entre las tres instancias de forma equitativa (Round Robin), lo cual es confirmado por el campo Hostname de la respuesta. Tercero, una prueba de rendimiento comparativa entre Traefik, Nginx y HAProxy como reverse proxy utilizando Apache Bench.

Finalmente hemos agregado una sección de extras donde podremos ver algunas implementaciones adicionales.

2. Objetivo y alcance

Objetivo

Construir y validar una solución mínima de arquitectura orientada a servicios utilizando Traefik, demostrando empíricamente su capacidad para enrutar tráfico HTTP, aplicar middlewares de transformación y ejecutar un balanceo de carga real sobre múltiples instancias de un servicio backend.

Alcance

Debido a la restricción de extensión máxima de este documento (10 páginas), el presente informe tiene un alcance acotado al cumplimiento técnico de las consignas solicitadas en el Trabajo Práctico. Se centra en la investigación de los aspectos más relevantes del software, la configuración de una infraestructura basada en HTTP (lo que no significa que Traefik no pueda gestionar tráfico bajo otro tipo de protocolos), el código de despliegue y la evidencia empírica del ruteo y el balanceo de carga.

Adicionalmente, (y por ello la extensión mayor del documento a 10 páginas) en la sección 7 se incluirán implementaciones complementarias (File Provider, healthchecks, enrutamiento a hosts externos con balanceo por pesos) que buscan respaldar con evidencia práctica los conceptos desarrollados en el marco teórico, aunque no se hayan solicitado en la consigna. Durante la presentación oral se profundizará en estos aspectos.

3. Desarrollo

3.1 ¿Qué es Traefik?

Básicamente, Traefik es un reverse proxy¹ y balanceador de carga de código abierto escrito en Go, cuya principal característica es la capacidad de configurarse de forma automática y dinámica en entornos contenerizados mediante el descubrimiento de servicios. Se integra con Docker y detecta nuevos contenedores al instante, levantando las reglas de enrutamiento directamente de sus etiquetas (labels²).

Docker, en este contexto, cumple el rol de Provider: Traefik se conecta al socket de Docker, escucha los cambios en los contenedores y actualiza las rutas automáticamente. Docker no es el único Provider³ ⇒ Si un ecosistema posee servicios no dockerizados, alojados en distintos hosts o con IPs externas, Traefik puede enrutar el tráfico hacia ellos integrando simultáneamente otros proveedores, como el File Provider (archivos de configuración dinámica en formato YAML o TOML).

Estas características le otorgan una notable facilidad operativa frente a alternativas como Nginx. Ambos permiten actuar como proxy inverso y balancear tráfico entre múltiples réplicas, pero difieren en el esfuerzo que esto implica. En Nginx, agregar réplicas requiere modificar manualmente la configuración y recargar el servicio. Traefik, en cambio, elimina esta complejidad: si la demanda exige aumentar las réplicas de un servicio de 3 a 10 instancias, detecta los nuevos contenedores a través del socket de Docker, lee sus labels, los incorpora al pool de balanceo de carga y comienza a derivarles tráfico sin intervención manual ni reinicios.

Finalmente, su versatilidad no se limita a HTTP: también soporta TCP y UDP, permitiendo gestionar y balancear la carga de APIs, bases de datos o servicios de streaming bajo un único punto de entrada unificado.

Para más información: <https://doc.traefik.io/traefik/v3.0/providers/overview/>

3.2 Casos de uso de Traefik

3.2.1 Traefik como Balanceador de Carga

Balanceo de carga es la técnica de distribuir el tráfico entrante entre múltiples instancias de un mismo servicio con el objetivo de optimizar el uso de recursos, maximizar la disponibilidad y minimizar los tiempos de respuesta.

El balanceador de carga de Traefik soporta múltiples estrategias para distribuir el tráfico entre las distintas instancias. Para nuestro proyecto utilizaremos **wrr - (Weighted Round Robin)**: Es la estrategia por defecto. Traefik reparte las peticiones por turnos exactos. Si tienes 3 instancias de Spring, la petición 1 va a la

¹ **Reverse Proxy (proxy inverso)**: Servidor intermediario que recibe las solicitudes de los clientes y las redirige al servicio backend correspondiente. El cliente nunca se comunica directamente con el servidor final, lo que permite centralizar funciones como balanceo de carga y control de acceso.

² **Labels (etiquetas)**: Metadatos clave-valor que se asignan a los contenedores Docker y que Traefik lee (Traefik define un conjunto específico de labels que el sabe interpretar) para determinar las reglas de enrutamiento, el dominio asociado, los middlewares a aplicar y otros parámetros de configuración de cada servicio.

³ **Provider**: En Traefik, un Provider es una fuente de configuración dinámica que Traefik consulta o escucha para descubrir servicios, rutas y reglas de enrutamiento. Ejemplos: Docker, Kubernetes, Consul, File Provider, entre otros.

Instancia A, la 2 a la B, la 3 a la C, la 4 vuelve a la A, y así sucesivamente. Tenemos la posibilidad de agregar pesos a los servicios: las instancias con mayor peso reciben proporcionalmente más tráfico. Útil cuando las instancias tienen distinta capacidad de hardware. Para más información diríjase a la sección referencias.

3.2.2 Traefik como Reverse Proxy

Traefik actúa como un punto de entrada único que gestiona y distribuye las peticiones externas hacia los servicios internos. Más allá del simple enrutamiento, permite implementar estrategias avanzadas de gestión de tráfico sin añadir complejidad al código de la aplicación.

Entre sus capacidades destacan el soporte para múltiples protocolos modernos (HTTP, gRPC, WebSockets) y la implementación de estrategias de despliegue como el uso de Middlewares. Estos últimos permiten añadir capas de seguridad y control, como límites de tasa (rate limiting), reintentos automáticos y manipulación de cabeceras, todo gestionado directamente desde el proxy.

3.2.3 Observabilidad

Traefik proporciona capacidades integradas de monitorización y observabilidad que permiten conocer el estado del sistema en tiempo real. A través de Logs, Access Logs (ver sección 6.3.2) y un Dashboard web integrado (ver sección 6.1.2), podemos identificar cuellos de botella, diagnosticar errores y supervisar el comportamiento de los servicios. El Dashboard ofrece una vista centralizada de los routers, services, middlewares y entypoints configurados, permitiendo verificar de un vistazo qué rutas están activas, qué servicios están respondiendo y qué middlewares se están aplicando.

Además, Traefik permite exportar métricas a proveedores externos como Prometheus, Datadog o InfluxDB, lo que posibilita visualizar y analizar datos de rendimiento, tasas de error y tiempos de respuesta a través de herramientas de visualización como Grafana.

Complementando estas herramientas, Traefik permite configurar healthchecks sobre los servicios backend. Mediante solicitudes periódicas a un endpoint definido (por ejemplo, /health), Traefik verifica de forma continua el estado de cada instancia. Si una instancia deja de responder o devuelve un error, Traefik la retira automáticamente del pool de balanceo de carga y redistribuye el tráfico entre las instancias saludables, reincorporándola cuando vuelve a estar operativa. Este mecanismo refuerza la alta disponibilidad del sistema sin requerir intervención manual.

3.2.4 Seguridad

Para la gestión segura del tráfico HTTPS, Traefik implementa TLS Termination⁴ : actúa como punto centralizado de descifrado, recibiendo la conexión HTTPS del cliente, desencriptándola y reenviándola en HTTP plano a los servicios internos. Esto libera a los contenedores de gestionar certificados propios; por ejemplo, si contamos con contenedores de Spring o Nginx detrás de Traefik, estos simplemente reciben tráfico HTTP sin necesidad de configurar TLS.

⁴ **TLS Termination:** Proceso mediante el cual el reverse proxy recibe la conexión cifrada (HTTPS) del cliente, la descifra, y reenvía la solicitud en texto plano (HTTP) a los servicios internos.

Traefik también permite proteger rutas mediante middlewares (ver sección 3.3 para comprender middlewares) de autenticación. BasicAuth y DigestAuth permiten restringir el acceso con usuario y contraseña, mientras que ForwardAuth delega la autenticación a un servicio externo (como Google Auth u otro proveedor de identidad): antes de dejar pasar la solicitud, Traefik consulta a ese servicio y solo si responde afirmativamente el tráfico continúa hacia el backend. Para más información dirigirse a la sección de referencias.

3.3 ¿Cuáles son sus componentes?

Traefik se organiza en cuatro componentes principales que trabajan en cadena: Entrypoints, Routers, Middlewares y Services.

EntryPoints: Son los puntos de entrada a Traefik. Definen el puerto y protocolo (TCP, UDP, HTTP) en el que Traefik escucha las conexiones entrantes. Se definen en la configuración estática (archivo traefik.yml) y no cambian en tiempo de ejecución. Los más habituales son el puerto 80 para HTTP y el 443 para HTTPS.

Routers: Son las reglas de enrutamiento. Cuando llega una petición a un Entrypoint, el Router la analiza y decide a qué Service enviarla. Las reglas se expresan mediante un DSL (Domain-Specific Language) propio de Traefik que permite evaluar condiciones como el host, el path de la URL, los headers HTTP o el método de la petición. Por ejemplo, una regla puede indicar que toda petición cuyo path comience con /api se dirija a un Service determinado. Los Routers también son el punto donde se asocian los Middlewares y la configuración TLS. En entornos Docker, se configuran mediante labels en los contenedores.

Middlewares: Son componentes que transforman o filtran las peticiones antes de que lleguen al Service destino, o las respuestas antes de devolverlas al cliente. Se encadenan en el Router y se ejecutan en orden. Traefik incluye una variedad de middlewares nativos; en este proyecto nos hemos centrado en StripPrefix, que elimina un segmento del path antes de reenviar la petición (por ejemplo, permite que /api/whoami llegue al servicio como /whoami). Esta capacidad permite desacoplar la estructura de enrutamiento externa de la lógica interna del microservicio. De este modo, el backend no necesita conocer la ruta bajo la cual ha sido expuesto, recibiendo únicamente el path que está diseñado para procesar. Ejemplo: Llega midominio.com/api/usuarios, actúa el Middleware, ve que el prefijo es /api y lo recorta y al backend solo le llega /usuarios.

Services: Representan los backends reales: los contenedores o servidores que procesan las peticiones. Es a nivel de Service donde se configura el balanceo de carga entre múltiples réplicas y los healthchecks (ver sección 3.2.3) para verificar la disponibilidad de cada instancia.

3.4 Comparativa con sistemas alternativos a Traefik

En la sección 6.3.2 podrá visualizar una comparativa de rendimiento entre los distintos sistemas.

3.4.1 Traefik vs Nginx vs HAProxy

Mientras que Nginx y HAProxy son soluciones robustas escritas en C que priorizan el rendimiento, la baja latencia, es cierto que tienen un control manual exhaustivo. Traefik apuesta por la automatización dinámica. La diferencia clave es que Nginx y HAProxy requieren configuraciones estáticas y herramientas externas para gestionar SSL o detectar cambios, mientras que Traefik se integra nativamente con la API de Docker/Kubernetes (y los demás providers) para auto-descubrir servicios y gestionar de forma autónoma. En resumen: Nginx y HAProxy son más eficientes en consumo de recursos para cargas masivas, pero Traefik reduce drásticamente la complejidad operativa en arquitecturas de microservicios que cambian constantemente.

Mientras que Traefik brilla en la flexibilidad del nivel de aplicación (Capa 7), HAProxy es el referente absoluto en la Capa 4 (TCP) y estabilidad de carga masiva. HAProxy ofrece algoritmos de balanceo mucho más granulares y una inspección de paquetes más profunda. La principal diferencia radica en que HAProxy es un componente "estático": cualquier cambio en el backend suele requerir una recarga o el uso de una API externa, mientras que Traefik es "nativo al cambio". Si la infraestructura es mayormente estática y el volumen de tráfico es crítico, HAProxy es superior; si la infraestructura cambia constantemente, Traefik reduce la carga administrativa.

3.5 Desventajas y Limitaciones.

→ Pese a su agilidad, Traefik presenta desafíos operativos importantes. El uso de etiquetas (labels) para la configuración puede derivar en "fallos silenciosos" (errores 404) ante simples errores tipográficos. El sistema Traefik continuará funcionando, pero el servicio mal configurado no.

→ En arquitecturas complejas, la acumulación de etiquetas ensucia la definición de la infraestructura (ej. docker-compose), volviéndola verbosa y a nuestro criterio difícil de entender y mantener. No permite separar las responsabilidades.

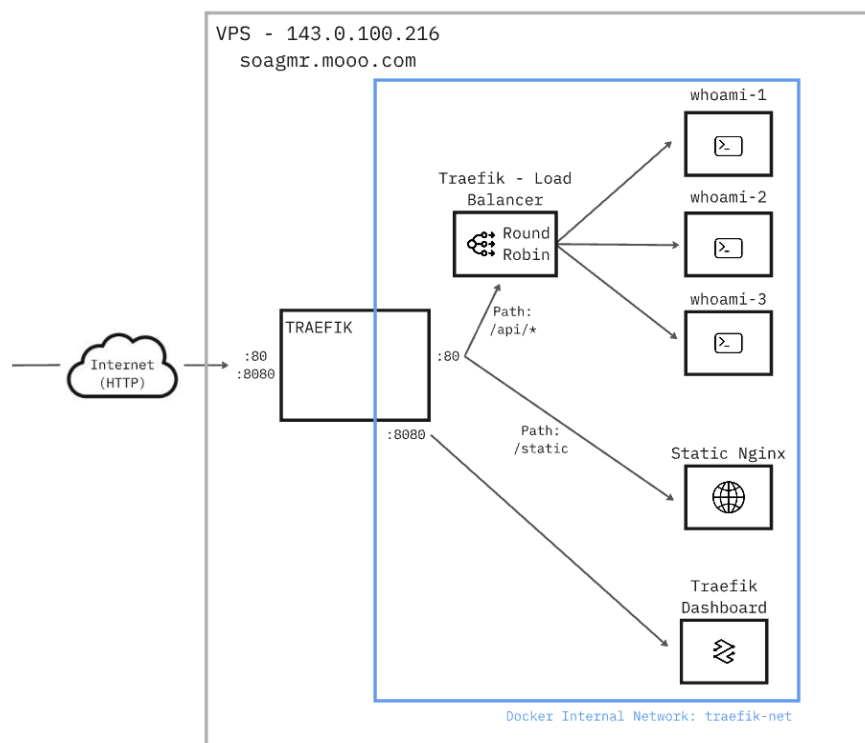
→ Al estar escrito en Go, su consumo de recursos es ligeramente superior al de alternativas en C bajo escenarios de estrés extremo.

→ La limitación más crítica en su versión *Open Source* es la falta de soporte nativo para TLS en Alta Disponibilidad (HA), obligando a implementar soluciones externas (como Consul o etcd).

4. Arquitectura

4.1 Diagrama Arquitectura

La arquitectura que se implementará para este trabajo partirá del siguiente esquema:



4.1.1 Visualización en el dashboard de Traefik

La configuración implementada puede visualizarse mediante el dashboard de Traefik, accesible a través del puerto 8080.

En esta interfaz se observan los componentes principales de la arquitectura, incluyendo routers, services y middlewares generados dinámicamente a partir de la configuración definida en Docker Compose.

4.2 Explicación Arquitectura

La arquitectura se despliega sobre la máquina virtual asignada para este trabajo. Traefik actúa como único punto de entrada (Edge Router), exponiendo los puertos 80 (tráfico HTTP) y 8080 (Dashboard de monitoreo). Todas las peticiones externas son interceptadas por Traefik, el cual inspecciona la ruta solicitada. Si la petición incluye el prefijo /api (independientemente de lo que continúe), se enruta hacia un clúster de tres instancias del servicio whoami a los que se les realizará balanceo de carga. Si la ruta coincide con /static, se dirige a un contenedor independiente con Nginx. Toda esta comunicación ocurre de forma aislada a través de una red privada de contenedores Docker denominada traefik-net, evitando exponer los puertos de los servicios internos directamente a Internet.

5. Implementación

5.1 docker-compose.yml

Para la implementación se utilizó un archivo docker-compose.yml, disponible en el repositorio de GitHub indicado en el anexo, en el archivo traefik-cp/docker-compose.yml.

Este archivo define todos los servicios necesarios para ejecutar la arquitectura: Traefik, el servicio API basado en whoami, el servicio static basado en Nginx y la red interna compartida.

La infraestructura se orquesta mediante Docker Compose, lo que permite levantar todos los contenedores con un único comando y mantener una configuración reproducible. Traefik actúa como punto único de entrada, exponiendo únicamente los puertos 80 para tráfico HTTP y 8080 para el dashboard de administración.

Una parte central de la configuración es el montaje del socket de Docker:

- `/var/run/docker.sock:/var/run/docker.sock:ro`

Este socket permite que Traefik consulte al motor de Docker y detecte automáticamente los contenedores en ejecución, sus etiquetas, redes y puertos internos. El modo ro indica que el socket se monta en solo lectura, reduciendo el riesgo de modificaciones desde el contenedor de Traefik.

Gracias a esta integración, Traefik utiliza el Docker Provider para realizar autodescubrimiento de servicios. Esto significa que los servicios backend, como whoami y nginx, no necesitan declararse manualmente en un archivo estático del proxy. En cambio, cada servicio define su comportamiento mediante labels de Docker.

Por ejemplo, las labels indican:

- si el servicio debe ser expuesto por Traefik
- qué regla de ruteo debe aplicarse
- qué endpoint utilizar
- qué middleware ejecutar
- qué puerto interno del contenedor debe recibir el tráfico

En esta implementación, el servicio whoami se publica bajo la ruta /api y cuenta con tres réplicas, permitiendo demostrar el balanceo de carga. Por su parte, el servicio static se publica bajo la ruta /static y sirve contenido HTML mediante Nginx.

Ambos servicios se conectan a una red interna llamada traefik-net y no exponen puertos directamente al host. De esta forma, todo el tráfico externo debe ingresar obligatoriamente a través de Traefik, cumpliendo con el requisito de punto único de entrada.

5.1.2 Balanceo de Carga

El balanceo de carga se aplica sobre el servicio whoami, el cual fue desplegado con 3 réplicas idénticas (replicas: 3). Al no especificarse pesos asimétricos en las etiquetas de configuración, el Service interno de Traefik aplica su algoritmo por defecto wrr.

5.1.3 Middleware

Se implementó el middleware StripPrefix en la ruta /api. Su función es interceptar y transformar la petición HTTP antes de entregarla al contenedor destino. Cuando un usuario solicita una URL como soagmr.mooo.com/api/pruebatp, el Router valida que el path

comience con /api y el middleware recorta ese prefijo antes de reenviar la petición al servicio. Como resultado, la instancia de whoami recibe una petición limpia apuntando a /pruebatp.

¿Para qué nos puede servir esto? Tal como hemos explicado en la sección 3.3, nosotros podemos tener servicios que internamente tienen endpoints tales como por ejemplo /, /perfil, /login. Ese servicio fue desarrollado sin saber que Traefik lo va a exponer bajo /api/users. Si llega una petición a /api/users/perfil y Traefik la reenvía tal cual, el servicio recibe /api/users/perfil y no sabe qué hacer con eso porque no tiene esa ruta devolviendo así 404. StripPrefix soluciona eso: quita /api/users y el servicio recibe /perfil, que sí conoce y puede responder.

6. Pruebas y evidencias

En esta sección se realizarán las pruebas y se evidenciará el funcionamiento correspondiente de cada una de las configuraciones. Podrá encontrar todas evidencias en el repositorio de github en la sección evidencias. Además, en el README podrá encontrar distintos comandos para ir comprobando el funcionamiento de cada una de las partes.

6.1 Funcionamiento Routers

6.1.1 Funcionamiento Routers por Path

Evidenciamos el funcionamiento del ruteo por path hacia /api y hacia /static a través del puerto 80 HTTP.

6.1.1.1 /api

```
curl -i http://soagmr.moood.com/api/
```

Lo que se ve en la respuesta es la salida de traefik/whoami, que es una imagen de Docker muy liviana creada por el equipo de Traefik específicamente para pruebas y demos. Lo único que hace es responder con información sobre la petición que recibió y el contenedor que la procesó.

6.1.1.2 /static

```
curl -i http://soagmr.moood.com/static/
```

Si lo abrimos en el navegador (más visual) podremos ver la página estática. Para ello hemos creado un archivo index.html en la VM (~/traefik/static-content/) y lo montamos dentro del contenedor mediante un volumen Docker.

6.1.2 Funcionamiento Routers por Puerto

En la sección anterior con curl hemos ingresado a través del puerto 80 y Traefik a través de la lectura del path nos ha redirigido al servicio correspondiente. Si ingresamos a través del puerto 8080, se abre el Dashboard de Traefik: <http://soagmr.moood.com:8080/>

6.2 Funcionamiento Middleware

Como se observa en las capturas, tanto `/api/pruebatp` como `/api/whoami` responden correctamente con un 200 OK, ya que ambas coinciden con la regla `PathPrefix(/api)` y el `StripPrefix` actúa sobre ellas. En cambio, al solicitar `/whoami` directamente (sin el prefijo `/api`), se obtiene un 404 Not Found porque no existe ningún Router configurado para esa ruta, lo que confirma que el servicio sólo es accesible a través del path `/api/*`. Si verifica la sección “GET /whoami HTTP/1.1” se comprueba que aunque mandemos la petición a `/api/whoami`, realmente lo que recibe el servicio es `/whoami` por el siguiente comando:

```
curl -i http://soagmr.moood.com/api/whoami
```

6.3 Funcionamiento Balanceo de Carga

6.3.1 Prueba con CURL y diferenciación por IP

Para corroborar el balanceo de carga, haremos repetidas llamadas a `/api` y demostraremos que la ip interna de ese servicio asignada por la red de docker va cambiando (debido a que cada `whoami` instanciado tiene su propia ip) y por lo tanto su `Hostname` también. Para ello utilizaremos el mismo curl para las 3 pruebas:

```
curl -i http://soagmr.moood.com/api
```

Se evidencia que están respondiendo las diferentes instancias de `whoami` y de manera secuencial (por la configuración de round robin). También podemos evidenciarlo mediante la variación del **Hostname** ⇒ Id del contenedor Docker que procesó la petición. Podemos también ejecutar un pequeño script de bash con un `for` para comprobar este balanceo de carga (ver README en repositorio).

6.3.2 Prueba con Access Logs

Otra forma de evidenciar esto es a través del sistema de Access Logs de Traefik. Aquí se puede visualizar el registro exacto de cada petición HTTP que procesa el proxy y para ello, debemos actualizar nuestro docker-compose agregando la siguiente configuración en el servicio de Traefik: `--accesslog=true`

Una vez hecho esto, veremos los logs con: `docker logs -f traefik` (en tiempo real). Para ver la evidencia dirigirse a la sección 6.3.2 en el repositorio.

6.3.2 Prueba de Rendimiento Comparativa

Esta prueba tiene como objetivo medir y comparar el throughput y latencia de Traefik, Nginx y HAProxy como reverse proxy sobre el mismo backend. Para esto se utilizará la herramienta Apache Bench. Para la configuración del test se realizó lo siguiente:

- Requests totales: 1000
- Conexiones concurrentes: 50
- Backend 3 réplicas de `traefik/whoami`

Comandos:

```

ab -n 1000 -c 50 http://localhost/api/whoami → Traefik (:80)
ab -n 1000 -c 50 http://localhost:8081/whoami → Nginx (:8081)
ab -n 1000 -c 50 http://localhost:8082/whoami → HAProxy (:8082)

```

Resultados:

Métrica	Traefik	Nginx	HAProxy
Requests/sec	685	1089	834
Latencia Media	72ms	45ms	58ms
Latencia p50	70ms	45ms	52ms
Latencia p90	107ms	60ms	109ms
Latencia p99	137ms	62ms	132ms
Latencia Maxima	145ms	109ms	144ms
Requests Fallidos	0	0	0

Nginx lidera en throughput (1089 req/s) y latencia media (45ms). HAProxy se posiciona en el medio (834 req/s, 58ms). Traefik registra el menor throughput y mayor latencia. La diferencia tiene una causa técnica, cuando Traefik recibe una request y lo manda al backend, antes de enviarlo agrega 7 headers HTTP, con el fin de que el backend sepa de dónde vino el request originalmente. Nginx y HAProxy en configuración básica no agregan todos esos headers, hacen el forward de forma más liviana. El servicio whoami devuelve en el body todo lo que recibió, incluidos esos headers. Debido a eso, la respuesta de Traefik pesa 383 bytes y la de Nginx y HAProxy 170-201 bytes. Adicionalmente, por cada request en Traefik atraviesa un pipeline de evaluación de reglas (Router) y ejecución del middleware StripPrefix antes de llegar al backend, procesamiento que Nginx y HAProxy no realizan en esta configuración. Sin embargo, los tres proxies completaron los 1000 requests sin errores, lo que indica que la diferencia observada no es de estabilidad.

7. Problemas/Soluciones - Extras

Vamos a aprovechar esta sección para realizar algunas pruebas más allá de lo solicitado en el trabajo práctico, para evidenciar el funcionamiento de algunos conceptos que se han mencionado en el Marco Teórico y a lo largo de todo el trabajo, que consideramos importantes demostrar. Para ver las implementaciones, el código y las evidencias dirigirse al repositorio a la sección 7. Revisar el README.

7.1 Autodescubrimiento y File Provider: configuración dinámica sin reinicio

En la sección teórica (3.1) mencionamos que Docker no es el único Provider de Traefik y que es posible integrar simultáneamente otros proveedores, como el File Provider. En esta demostración lo comprobamos en la práctica.

Se levantó un contenedor whoami-file dentro del mismo docker-compose pero sin labels de Traefik, lo que significa que el Docker Provider lo ignora por completo. Inicialmente, al hacer

curl http://soagmr.mo00.com/file/, Traefik responde con 404 ya que no tiene ninguna ruta configurada hacia ese servicio.

```
bruno@B550m:~$ curl http://soagmr.mo00.com/file/
404 page not found
```

A continuación, sin detener ni reiniciar Traefik, se editó el archivo dynamic.yml agregando la configuración del router, middleware y servicio correspondiente. Gracias a la opción watch=true, Traefik detectó el cambio en el archivo y aplicó la nueva configuración automáticamente. Al repetir el curl, el servicio respondió con 200.

Veamos que se ha configurado:

- 1) En el docker-compose en el servicio traefik se ha agregado el file provider.
- 2) Montamos el volumen.
- 3) Agregamos un nuevo servicio separado de whoami en el mismo docker compose.
- 4) Creamos el archivo dynamic.yml.

Esta configuración es exactamente la misma que tenemos configurada en los labels de los otros servicios, pero escrita en formato YAML en vez de labels. La misma configuración escrita en formato labels para docker sería la siguiente:

```
traefik.http.routers.api-router.rule=PathPrefix(`/api`)
traefik.http.routers.api-router.service=api-service
traefik.http.routers.api-router.middlewares=api-strip
```

Si nos dirigimos a la consola de Traefik podemos ver que la configuración se ha implementado (por favor primero realizar los cambios en la maquina virtual indicados en el repositorio). Para nuestro caso:

<http://soagmr.mo00.com:8080/dashboard/#/http/routers/whoami-file-router@file>

7.2 HealthChecks

Traefik permite configurar healthchecks sobre los servicios backend mediante labels. El healthcheck consiste en una petición periódica a un endpoint del servicio (por ejemplo /health) para verificar que está operativo. En un backend real (Spring, Express, etc.), este endpoint se programa para devolver 200 OK si todo funciona correctamente, o un error si algo falló. En nuestro caso, whoami no tiene un endpoint /health específico pero responde 200 en cualquier ruta, por lo que funciona igualmente.

Configuramos Traefik para que cada 10 segundos realice esta verificación en cada réplica de whoami y, si no responde en 3 segundos, la retire del pool de balanceo. Para ello, agregamos los siguientes labels al servicio de whoami:

```
traefik.http.services.api-service.loadbalancer.healthcheck.path=/health
traefik.http.services.api-service.loadbalancer.healthcheck.interval=10s
traefik.http.services.api-service.loadbalancer.healthcheck.timeout=3s
```

Para demostrarlo, simulamos la caída de una réplica mediante docker pause (docker pause <ID_DEL_CONTENEDOR>), lo que provoca que el contenedor siga existiendo pero deje de responder. Sin healthcheck, Traefik seguiría enviándole tráfico y el cliente recibiría respuestas fallidas en una de cada tres peticiones. Con healthcheck configurado, Traefik

detecta que la instancia no responde, deja de derivarle tráfico y distribuye las peticiones únicamente entre las réplicas saludables. Al ejecutar `docker unpause`, Traefik la reincorpora automáticamente al pool.

Traes pausar uno de las instancias de whoami (pausaremos 002208369f8e ⇒ verificar `docker ps` en evidencias en github > ver en evidencias). Sin tener configurado el healthcheck, cuando hacemos `curl` y el balanceador de carga enruta hacia ese contenedor pausado no responde.

Tras configurar el healthcheck, veremos que nunca se enruta hacia el contenedor pausado y siempre tenemos a alguien que responda ante `/api`. Debería responder 1c4b890b3e19 y a840cbba3fb7.

Es importante destacar que si un contenedor se apaga por completo, Docker le notifica a Traefik a través del socket y lo elimina del pool sin necesidad de healthcheck. El healthcheck cubre un escenario distinto: cuando el contenedor sigue corriendo pero dejó de funcionar correctamente (un deadlock, una conexión a base de datos caída, etc.). Sin healthcheck, Traefik no tiene forma de detectar este tipo de fallas.

En el dashboard podemos ver que una de las replicas tiene problemas (porque está pausada):

<http://soagmr.moood.com:8080/dashboard/#/http/services/api-service@docker>



Servers	
STATUS	URL
	http://172.18.0.4:80
	http://172.18.0.8:80
	http://172.18.0.9:80

7.3 Enrutamiento a distintos host con balanceo de carga por prioridad

Para enrutar tráfico hacia hosts externos necesitamos utilizar el File Provider. Los labels solo funcionan para contenedores Docker, ya que Traefik los descubre a través del Docker Provider. Si el destino es una IP externa o un servidor que no es un contenedor, no hay contenedor donde poner el label, con lo cual necesitamos indicarle a Traefik la URL del backend explícitamente en el archivo de configuración dinámica.

Para demostrarlo, configuramos en `dynamic.yml` una ruta hacia `httpbin.org`, un servicio público externo que no forma parte de nuestra infraestructura Docker.

El router externo-router captura toda petición cuyo path comience con `/externo` y la dirige al `service externo-service`. Antes de reenviarla, el `middleware strip-externo` quita el prefijo `/externo` del path para que el host de destino reciba la ruta limpia (de lo contrario, `httpbin` recibiría `/externo/get` y devolvería 404 al no reconocer esa ruta). El service define el destino dentro de `loadBalancer`, que en este caso apunta a una única URL externa: `http://httpbin.org`.

Al hacer `curl http://soagmr.moood.com/externo/get`, Traefik intercepta la petición, aplica el `StripPrefix` y la reenvía a `http://httpbin.org/get`. El servicio externo responde con un JSON que incluye los datos de la petición recibida, confirmando que Traefik está enrutando correctamente hacia un host fuera de la infraestructura Docker.

Si quisiéramos configurar un balanceador de carga con pesos haríamos lo siguiente, modificando el archivo anterior (verificar `/traefik/traefik-tp/7`.

Problemas-Soluciones-Extras/7.3 Enrutamiento a distintos host con balanceo de carga por prioridad/balanceador de carga con pesos/dynamic.yml).

La diferencia principal con la configuración anterior es que el router ahora apunta a un servicio de tipo weighted en lugar de un loadBalancer directo. Este servicio weighted actúa como distribuidor: define dos servicios hijos (externo-primary y externo-secondary) con pesos 3 y 1 respectivamente, lo que significa que de cada 4 peticiones, 3 se dirigen al primario y 1 al secundario. Cada servicio hijo es un loadBalancer convencional que apunta a su host de destino.

Para probarlo, simplemente hacemos varios curl `http://soagmr.mooo.com/externo/get`. De cada 4 peticiones, 3 deberían responder con el JSON de `httpbin` y 1 debería dar `bad gateway` (porque pusimos una url random). Eso justamente demuestra que el tráfico se está repartiendo entre dos hosts distintos.

8. Conclusión

La implementación validó a Traefik como una solución eficiente para arquitecturas de microservicios, destacando su capacidad de descubrimiento automático y gestión dinámica de rutas y middlewares sin intervención manual. Las pruebas de balanceo confirmaron una distribución equitativa del tráfico mediante Round Robin, mientras que el uso de múltiples providers y healthchecks demostró su alta disponibilidad y flexibilidad para integrar servicios externos.

Si bien la comparativa de rendimiento posicionó a Traefik con una latencia ligeramente superior frente a Nginx y HAProxy, se concluye que este margen es un costo aceptable frente a la drástica reducción de la complejidad operativa y la agilidad que aporta en entornos elásticos y automatizados.

9. Anexo

Github:

- <https://github.com/agusclap/traefik-tp.git>

Presentación:

- https://www.canva.com/design/DAHHr6XS34o/1I7YMc_121fjgfwHMC07oA/edit

10. Referencias

File Provider

<https://doc.traefik.io/traefik/reference/install-configuration/providers/others/file/>

Balanceo de Carga:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/load-balancing/service>

TLS:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/tls/overview/>

HealthChecks:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/load-balancing/service>

Basic Auth:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/basicauth/>

Forward Auth:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/forwardauth/>

Digest Auth:

<https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/digestauth/>

Logs & Access Logs

<https://doc.traefik.io/traefik/observe/logs-and-access-logs/>

Metrics

<https://doc.traefik.io/traefik/observe/metrics/>

Tracing

<https://doc.traefik.io/traefik/observe/tracing/>

StripPrefix

<https://doc.traefik.io/traefik/reference/routing-configuration/http/middlewares/striprefix/>

Dashboard

<https://doc.traefik.io/traefik/reference/install-configuration/api-dashboard/>

Labels Traefik - Docker

<https://doc.traefik.io/traefik/reference/routing-configuration/other-providers/docker/>

httpbin:

<https://kennethreitz.org/software/websites/httpbin>

Documentación Nginx

<https://nginx.org/en/docs/>

Documentación HAProxy V.3.3.6

<https://docs.haproxy.org/3.3/intro.html>